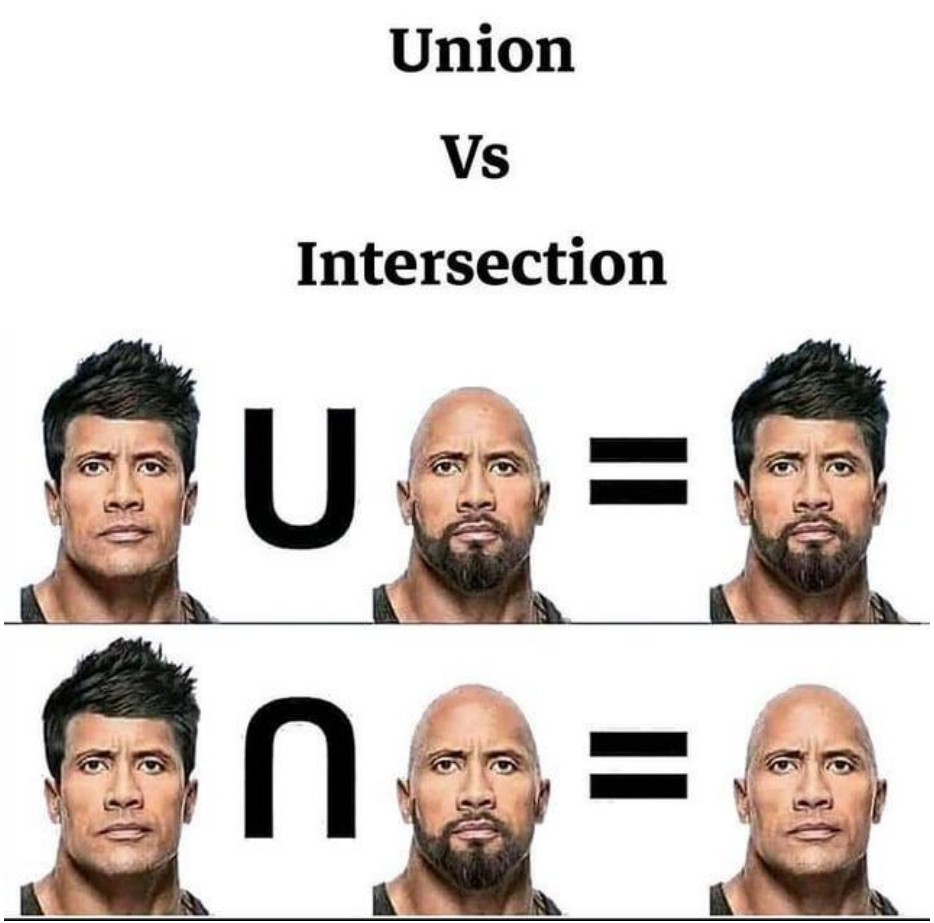
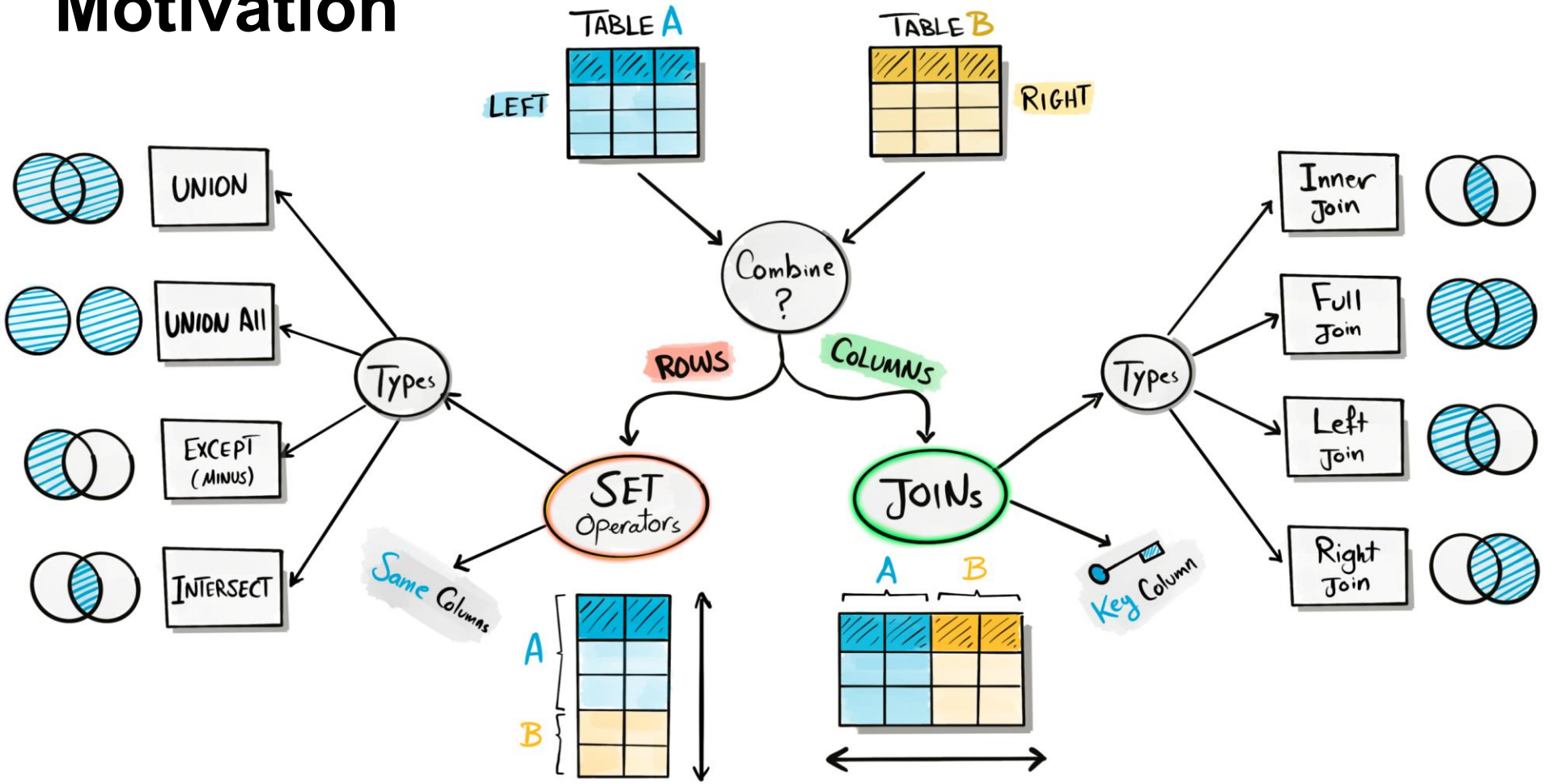
SQL JOIN Types

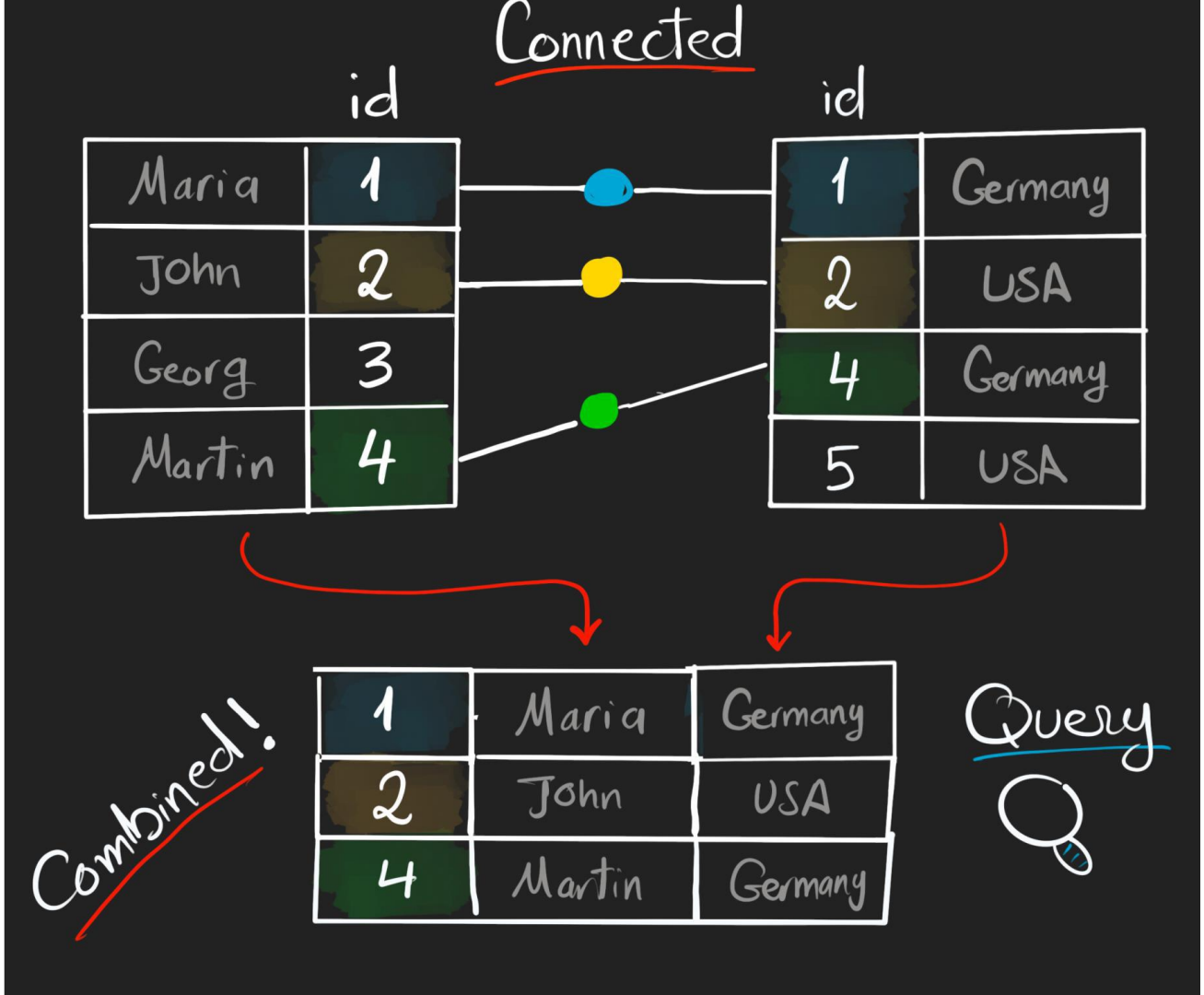
Theoretical and Practical Guide



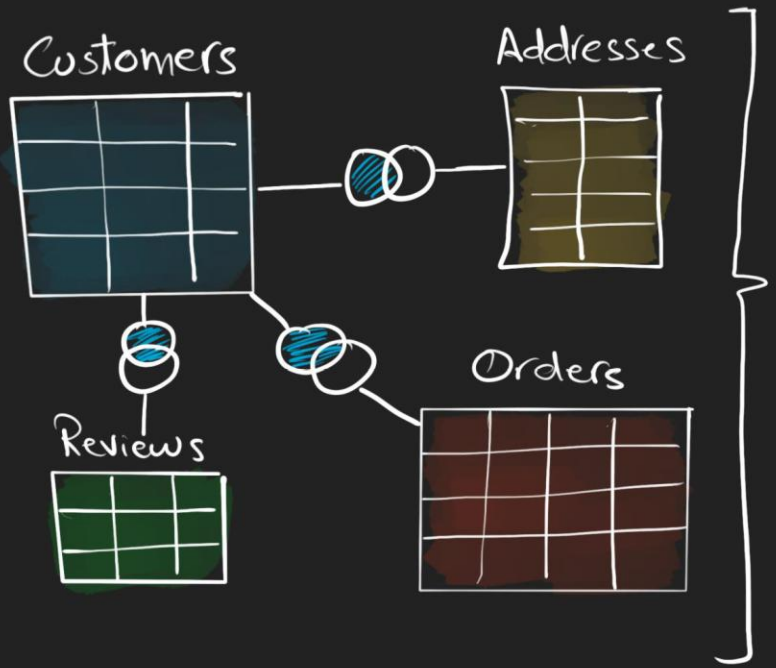
Motivation


TABLE A
LEFT
TABLE B
RIGHT
Combine ?
ROWS
COLUMNS
SET Operators
JOINs
Types
UNION
UNION All
EXCEPT (MINUS)
INTERSECT
Same Columns
A
B
Types
Inner Join
Full Join
Left Join
Right Join
Key Column
A
B

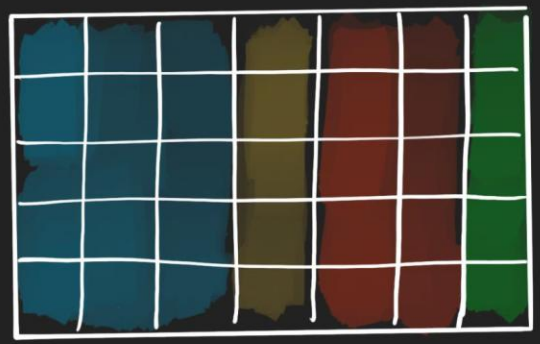
What is SQL JOIN?



1 Recombine Data

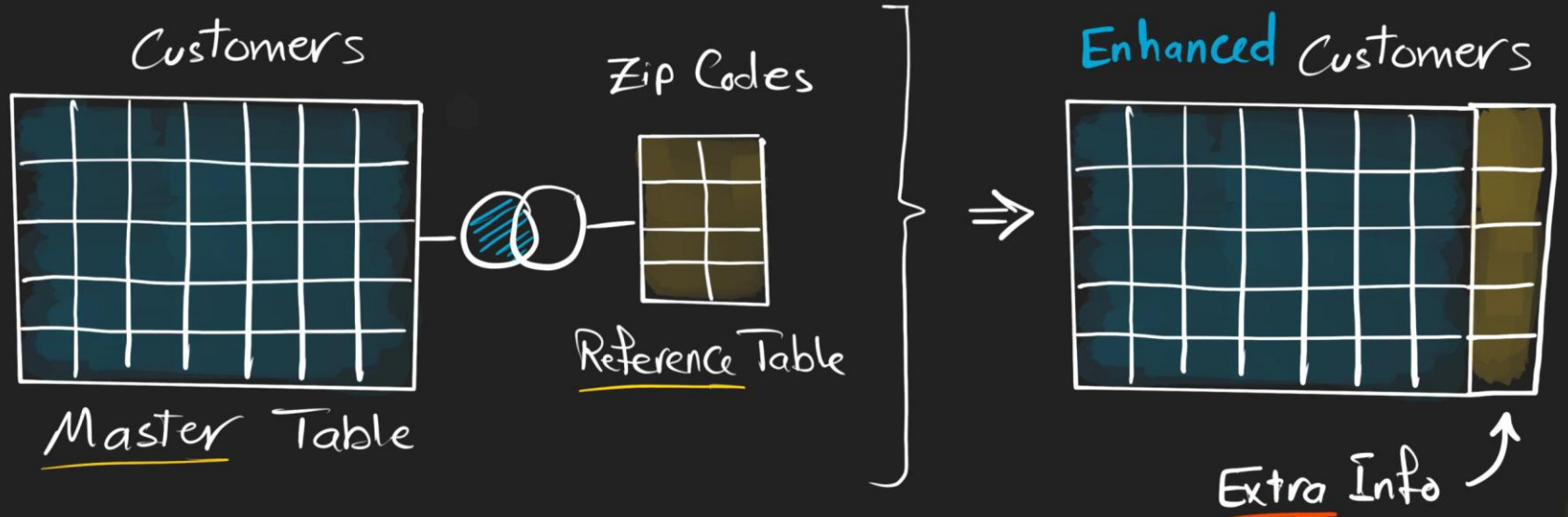


Complete Big Picture!

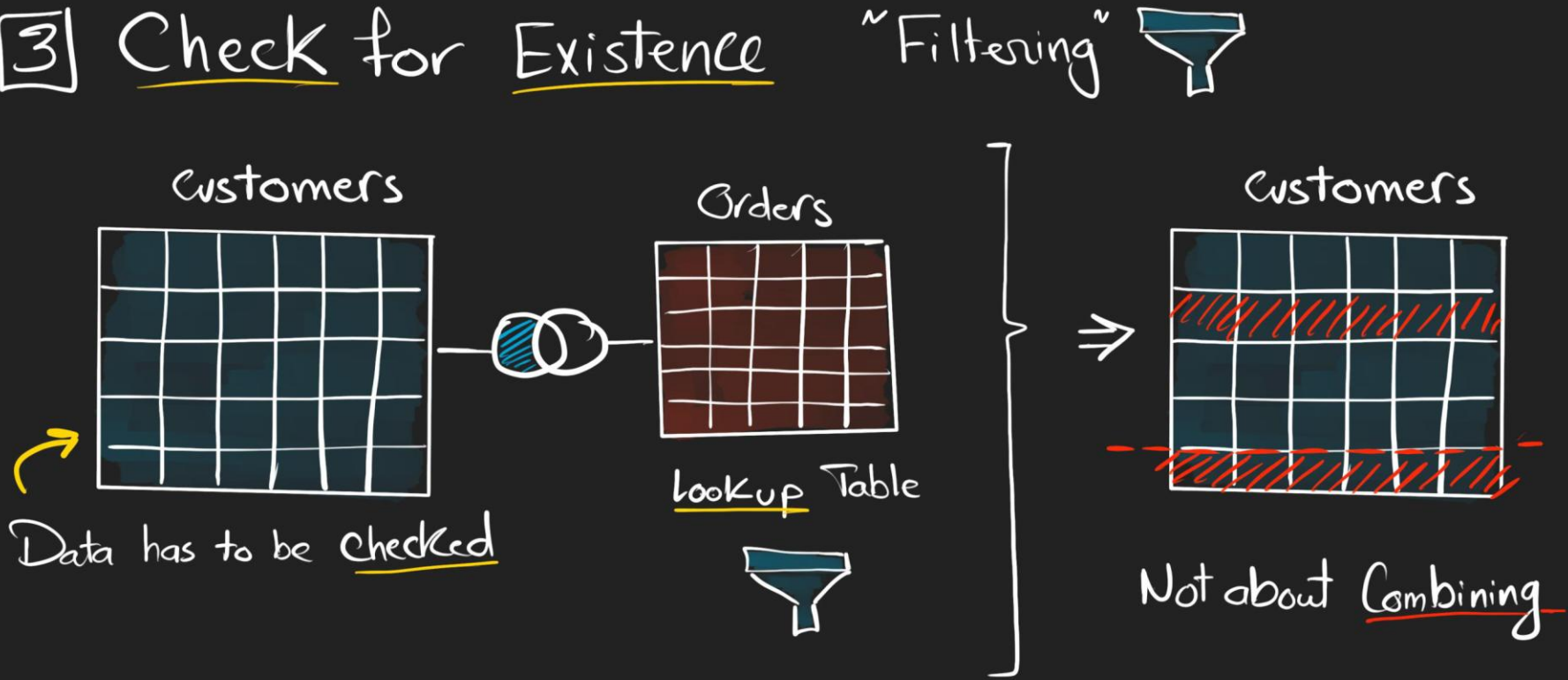


All Customers Details in One

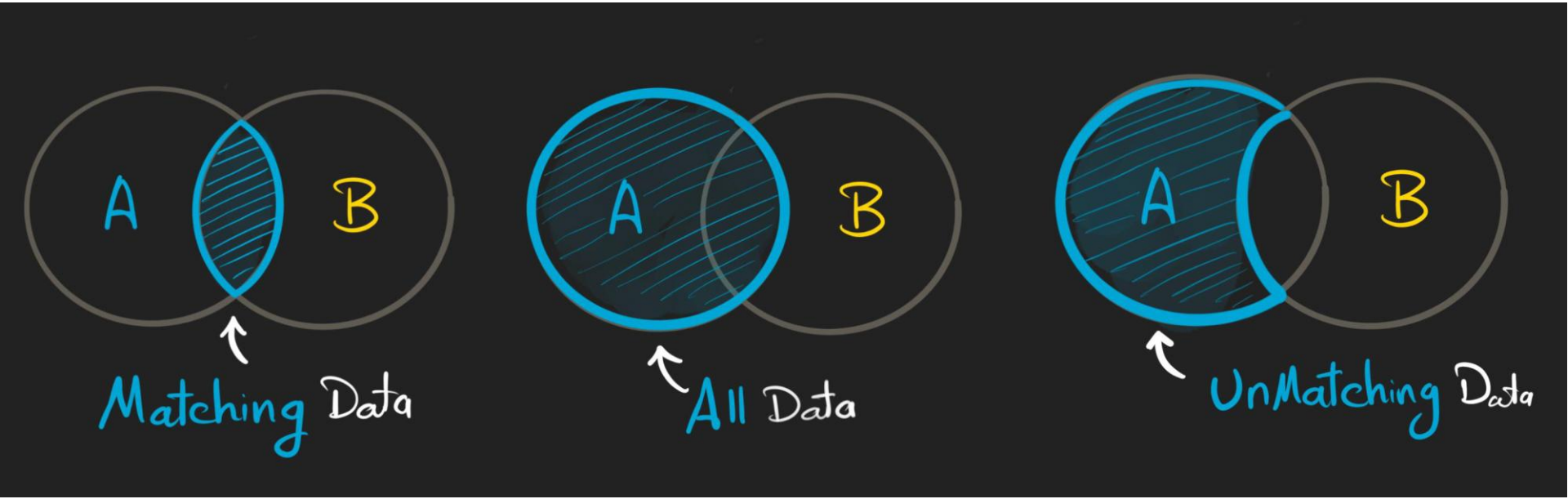
2 Data Enrichment "Getting Extra Data"



[3] Check for Existence "Filtering"



JOINs Possibilities



00. No JOIN

Returns Data from Tables without Combining Them



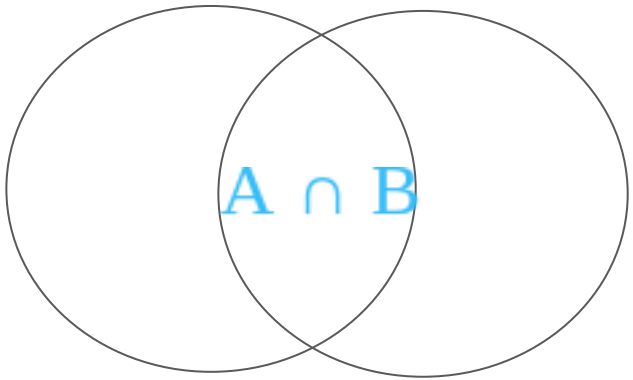
Two Results No Need To Combine

```
SELECT *
FROM A;

SELECT *
FROM B;
```

01. Inner Join: The Intersection

- **Concept:** Looks for the exact match between two tables.
- **Behavior:** Only returns records with pairs in both tables. It excludes any row without a relationship. Order is NOT important.
- **Formula:** $A \cap B$



Inner Join

Inner Join: Example Problem

Table A: Students

id_student	Name
1	Ana
2	Luis
3	María
4	Carlos

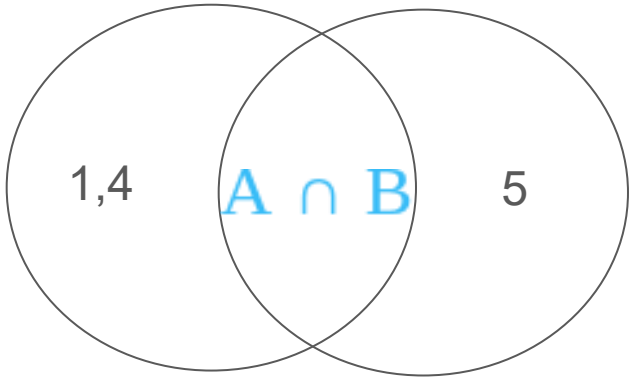


Table B: Enrollments

id_student	Course
2	Mathematics
3	History
3	English
5	Physics

Id_student	Name	Course
2	Luis	Mathematics
3	Maria	History
3	Maria	English

Table_A_students.csv
Table_B_enrollments.csv

```
SELECT a.id_student, a.name, b.course
FROM Students AS a
INNER JOIN Enrollments AS b
ON a.id_student = b.id_student;
```

INNER JOIN Use Case

The academic department wishes to know exactly the number of students who are at least enrolled once.

Problem Statement

Write an SQL query that displays:

- id_student
- name
- course

Only show those students that appear in both tables.

Expected Results

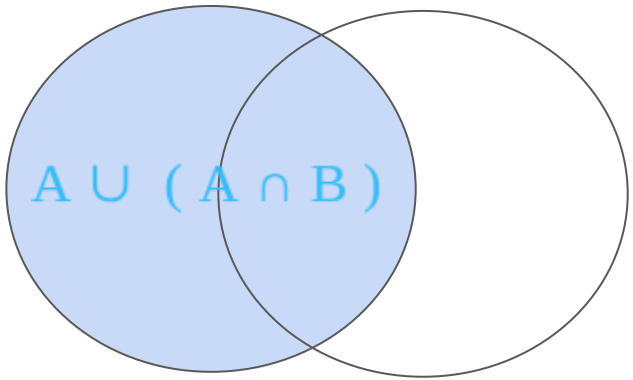
- Students without enrollment should not appear.
- A student could appear multiple times if he/she is enrolled in many courses.

02. **Left JOIN**: Left priority

- **Concept:** Maintains all information from the original (left) table.
- **Behavior:** Preserves the complete table A. Fills with NULL where there is no match in table B. Order is important.
- Formula:

$$A \cup (A \cap B)$$

"Show me ALL records from the left table, whether or not they have a match in the right table".



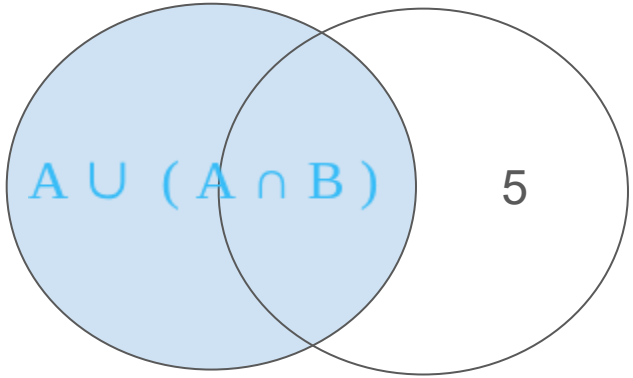
Left JOIN

Left JOIN: Example Problem

Table A: Students

Id_student	Name
1	Ana
2	Luis
3	María
4	Carlos

Table_A_students.csv
Table_B_enrollments.csv



Id_student	Name	Course
1	Ana	NULL
2	Luis	Matemathics
3	Maria	History
3	Maria	English
4	Carlos	NULL

Table B: Enrollments

id_student	Course
2	Mathematics
3	History
3	English
5	Physics

```
SELECT a.id_student,
    a.name,
    b.course
FROM Students AS a
LEFT JOIN Enrollments AS b
ON a.id_student = b.id_student;
```

LEFT JOIN Use Case

Problem Statement

The academic direction needs to identify which students are registered in the institution, regardless of whether they have courses enrolled.

Write an SQL query that displays:

- id_student
- name
- course

Show all students from table *Students*.

Expected Results

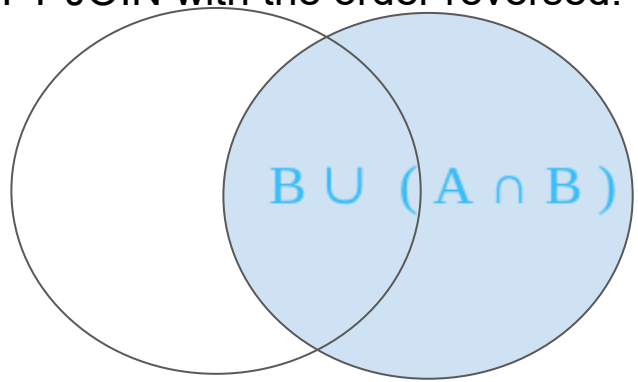
- If a student is not enrolled in any courses, then the field “Courses” should appear as NULL.
- How many students have not been enrolled in any course?

03. RIGHT JOIN: Right priority

- **Concept:** Prioritizes the right table (B) over the left (A).
- **Behavior:** Useful when table B is your master source of facts. It is less frequent but vital in audits. Order is important. Equivalent to a LEFT JOIN with the order reversed.
- **Formula:**

$$B \cup (A \cap B)$$

"Show me ALL records from the right table, whether or not they have a match in the left table."



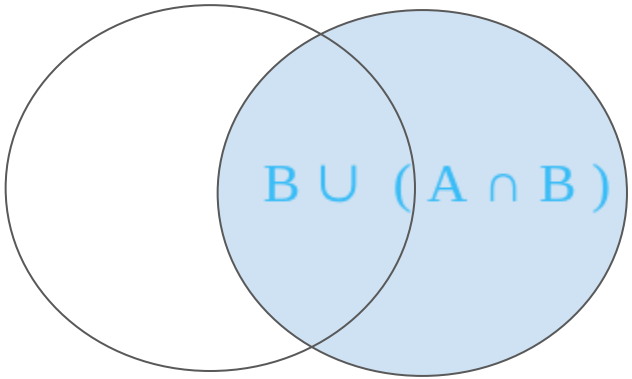
Right JOIN

Right JOIN: Example Problem

Table A: Students

Id_student	Name
1	Ana
2	Luis
3	Maria
4	Carlos

Table_A_students.csv
Table_B_enrollments.csv



Id_student	Name	Course
2	Luis	Mathematics
3	Maria	History
3	Maria	English
5	NULL	NULL

Table B: Enrollments

id_student	Course
2	Mathematics
3	History
3	English
5	Physics

```
SELECT a.id_student,
a.name,
b.course
FROM Students AS a
RIGHT JOIN Enrollments AS b
ON a.id_student = b.id_student;
```

RIGHT JOIN Use Case

Problem Statement

The systems department suspects there are enrollments registered for students who do not exist in the main student database.

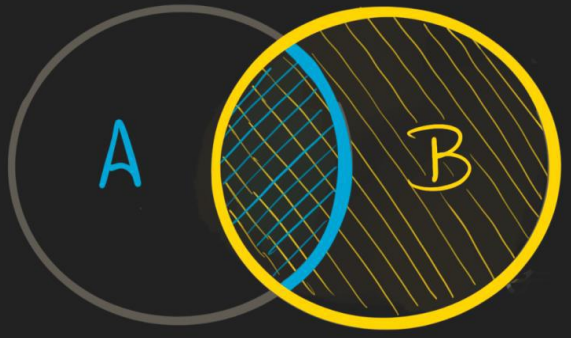
Write an SQL query that displays:

- id_student
- name
- course

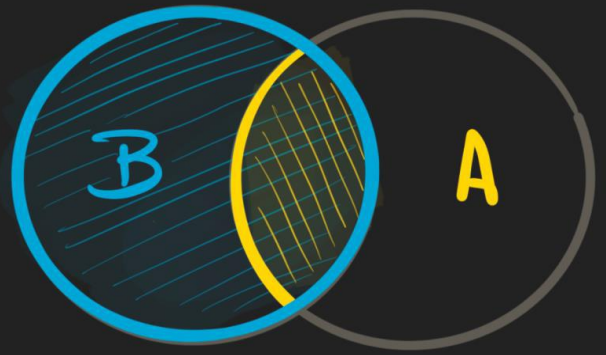
Show all rows from the table *Enrollments*.

Expected results

- Identify the values in “names” that appeared as NULL.



Same Results




```
SELECT *
FROM A
RIGHT JOIN B
ON A.Key = B.Key
```

Alternative ⇒

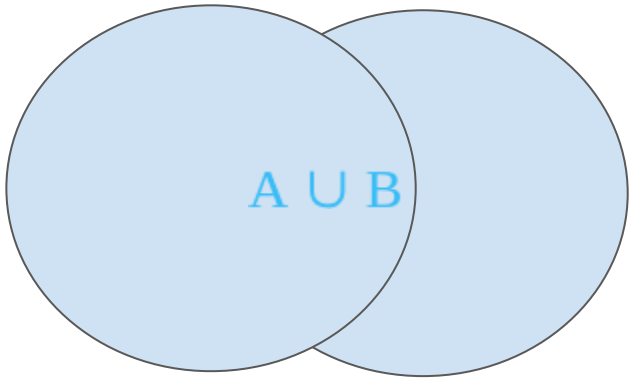
```
SELECT *
FROM B
LEFT JOIN A
ON A.Key = B.Key
```

04. **Full JOIN**: Total Union

- **Concept:** Nothing is lost. It combines records from both tables regardless of whether there is a match.
- **Behavior:** Ideal for integrity diagnostics. Generates NULLs in both directions. **Order is not important.**
- Fórmula:

$$A \cup B$$

“FULL OUTER JOIN shows ALL records from both tables, whether they match or not.”

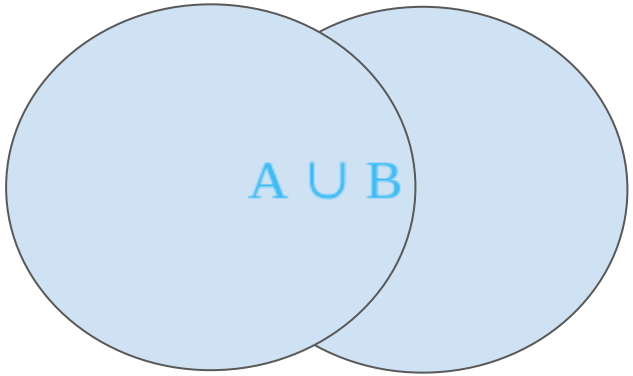


Full JOIN

Full Join: Práctica CSV

Table A: Students

Id_student	Name
1	Ana
2	Luis
3	María
4	Carlos



Id_student	Name	Course
1	Ana	NULL
2	Luis	Mathematics
3	Maria	History
3	Maria	English
4	Carlos	NULL
5	NULL	Physics

Table B: Enrollments

id_student	Course
2	Mathematics
3	History
3	English
5	Physics

Table_A_students.csv
Table_B_enrollments.csv

```
SELECT a.id_student,
a.name,
b.course
FROM Students AS a
FULL OUTER JOIN Enrollments AS b
ON a.id_student = b.id_student;
```

FULL OUTER JOIN Use Case

Problem Statement

The institution wishes to perform a complete audit to verify inconsistencies between tables.

Write an SQL query that displays all registers from both tables.

Expected Results

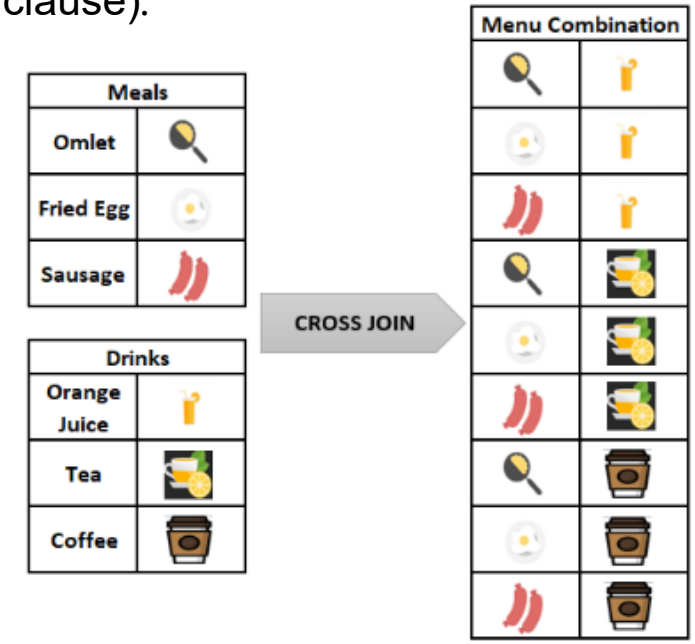
Identify:

1. Unenrolled students.
2. Enrollments with no students enrolled.

05. Cross JOIN: Combinatorics

- **Concept:** Generates the Cartesian Product: all rows of A with all rows of B.
- **Behavior:** It does not require a common key (ON clause).
- **Warning:** The size grows exponentially.
- Formula:

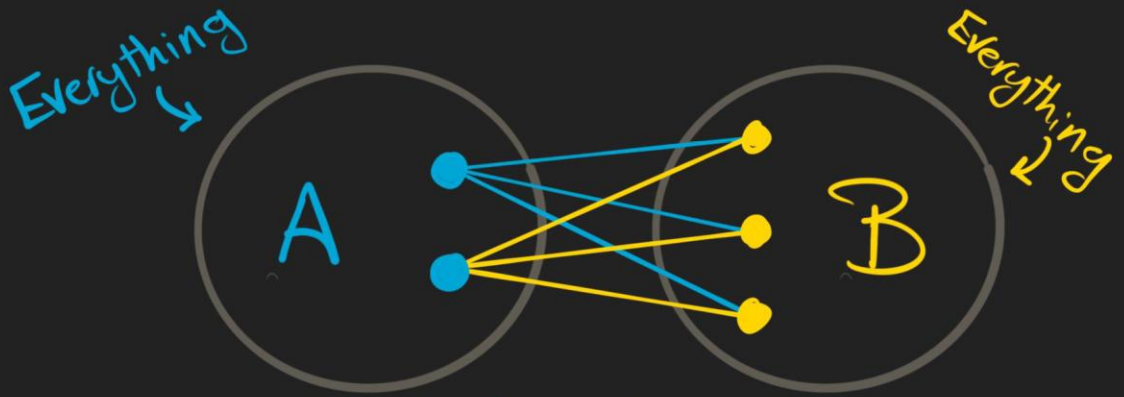
$$A \times B$$



CROSS JOIN

Combines Every Row from Left with Every Row from Right

All Possible Combinations - Cartersian Join -



2 x 3 = 6 ← Total Rows

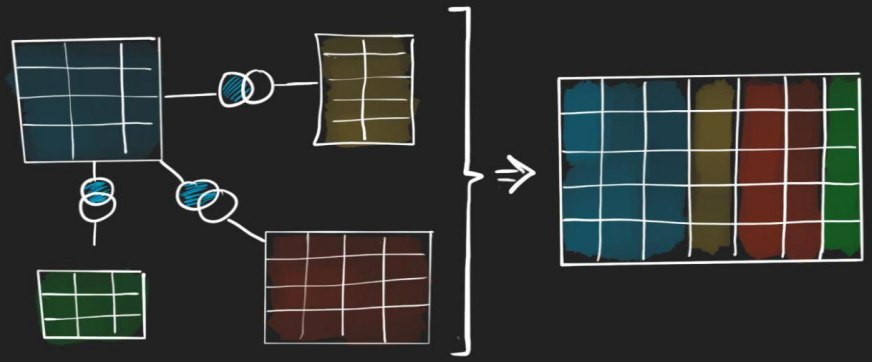
The Order of Table Doesn't Matter

```
SELECT *
FROM A
CROSS JOIN B
```

No Condition Needed

1 Recombine Data

~Big Picture~



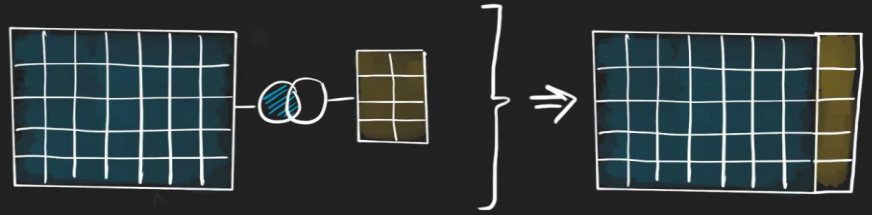
INNER

LEFT

FULL

2 Data Enrichment

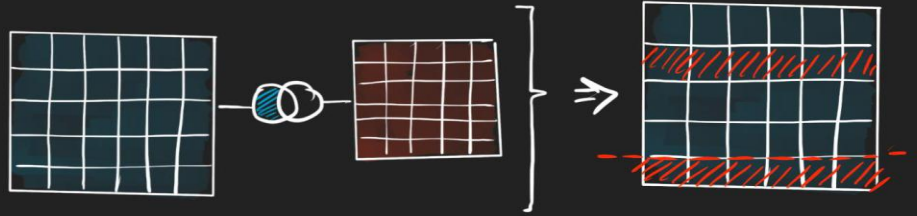
~Extra Info~



LEFT

3 Check Existence

~Filtering~

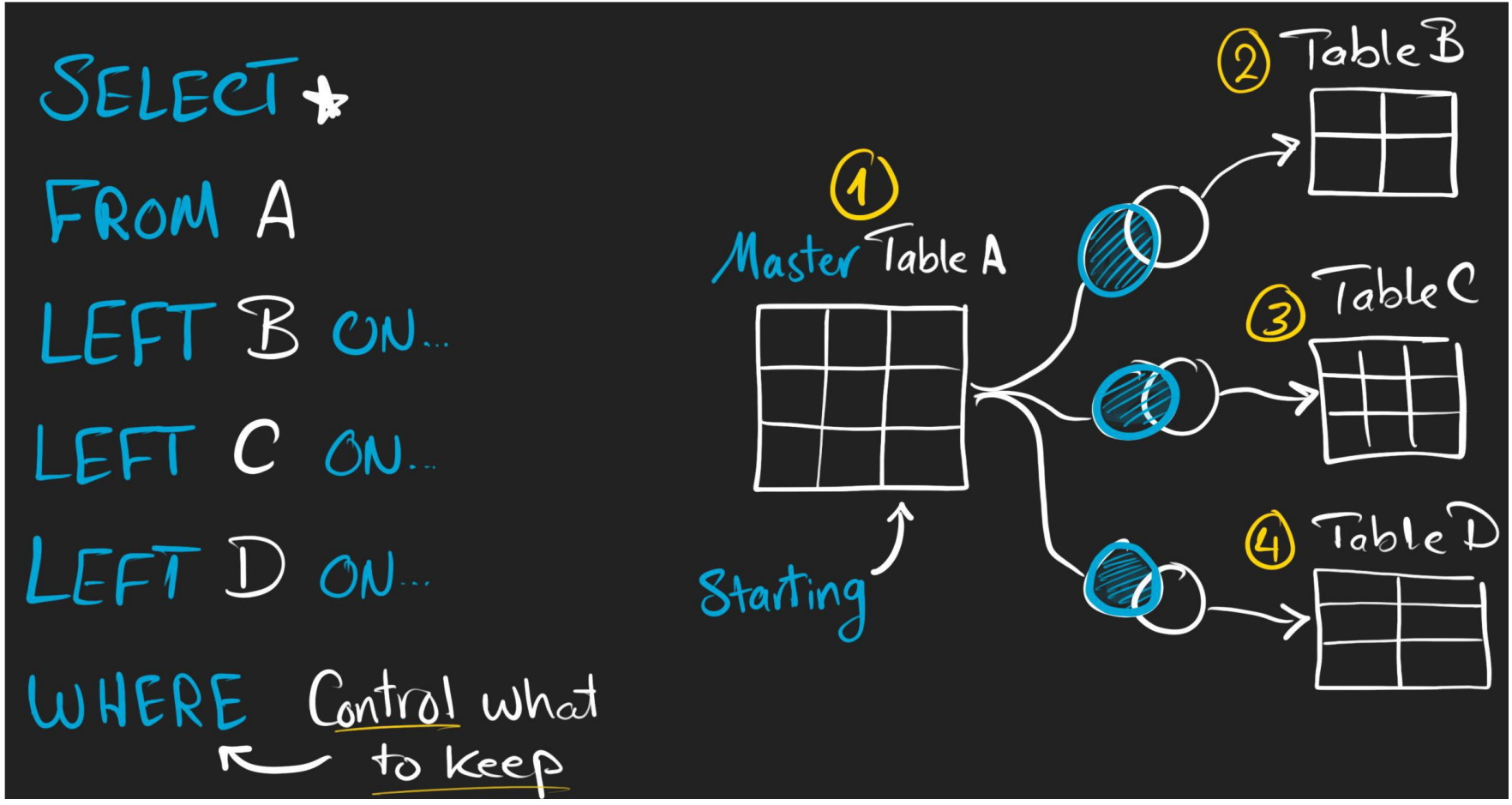


INNER

LEFT + WHERE

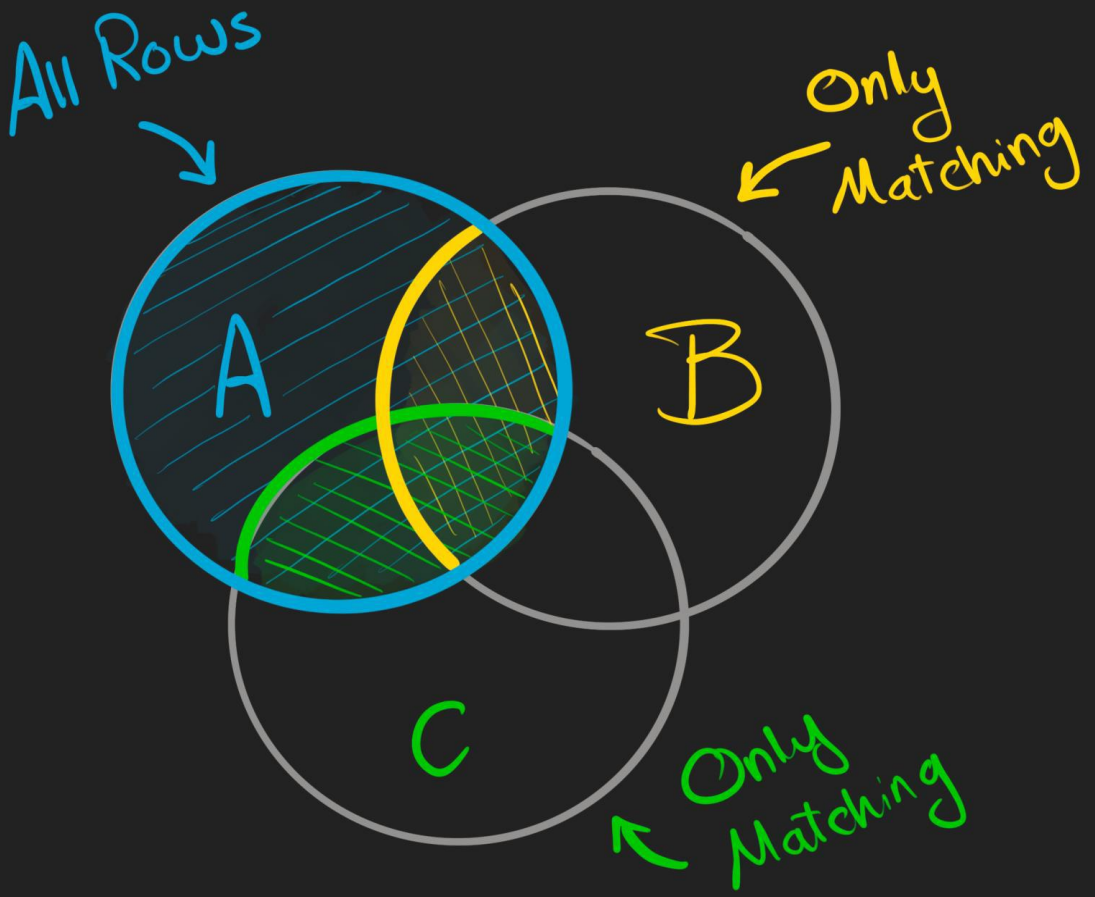
FULL + WHERE

How I JOIN Multiple Tables



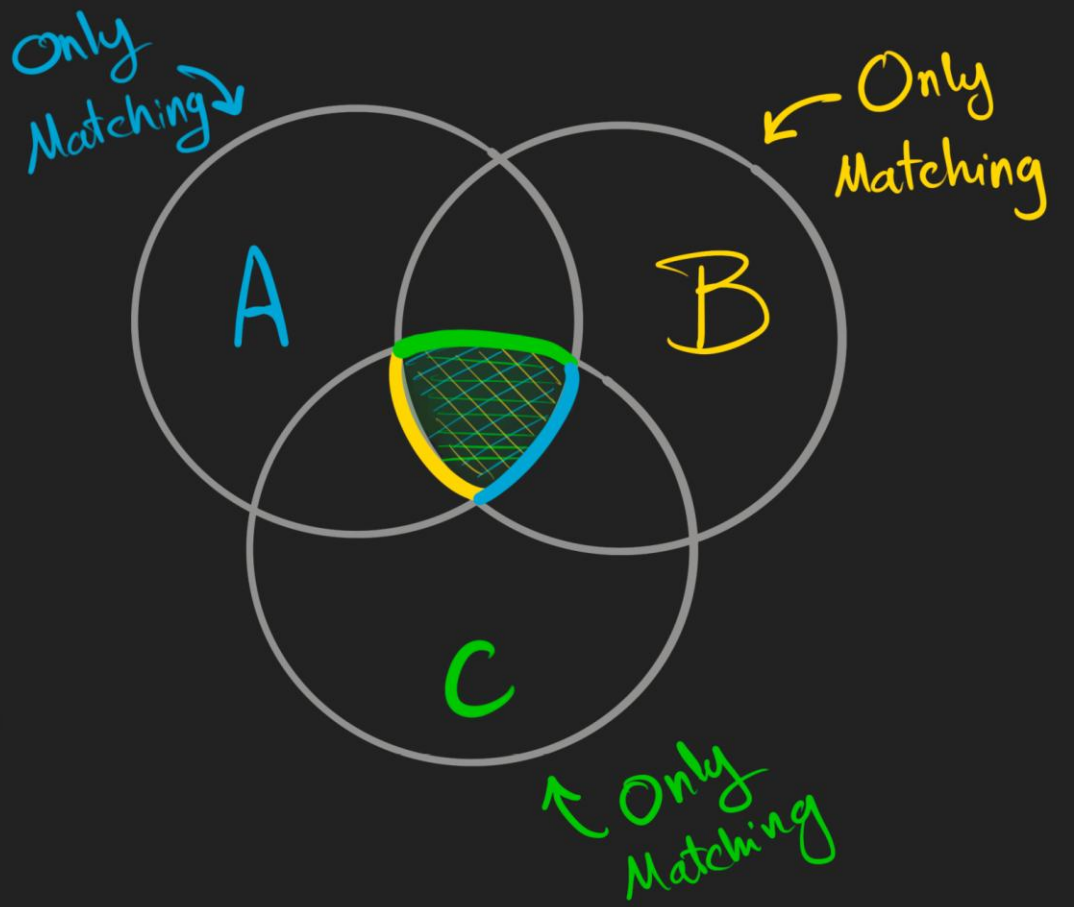
How I JOIN Multiple Tables

```
SELECT *
FROM A
LEFT B ON...
LEFT C ON...
LEFT D ON...
WHERE Control what to keep
```



INNER JOIN Multiple Tables

```
SELECT *
FROM A
INNER B ON...
INNER C ON...
⋮
```



EXERCISE

SQL JOINS - SLIDES

1. Create the Customers table first (Parent table)

```
CREATE TABLE Customers (
    CustomerID INTEGER PRIMARY KEY,
    FirstName TEXT NOT NULL,
    LastName TEXT,
    Country TEXT,
    Score INTEGER
);
```

Customers.csv

2. Create the Orders table (Child table with Foreign Key)

```
CREATE TABLE Orders (
    OrderID INTEGER PRIMARY KEY,
    ProductID INTEGER,
    CustomerID INTEGER,
    SalesPersonID INTEGER,
    OrderDate TEXT,
    ShipDate TEXT,
    OrderStatus TEXT,
    ShipAddress TEXT,
    BillAddress TEXT,
    Quantity INTEGER,
    Sales REAL,
    CreationTime TEXT,
    FOREIGN KEY (CustomerID)
REFERENCES Customers(CustomerID)
);
```

Orders.csv

3. Create the Employees table

```
CREATE TABLE Employees (
    EmployeeID INTEGER PRIMARY KEY,
    FirstName TEXT NOT NULL,
    LastName TEXT,
    Department TEXT,
    BirthDate TEXT, -- SQLite uses TEXT for dates
    Gender TEXT,
    Salary INTEGER,
    ManagerID INTEGER,
    FOREIGN KEY (ManagerID) REFERENCES
Employees(EmployeeID)
);
```

Employees.csv

4. Create the Products table

```
CREATE TABLE Products (
    ProductID INTEGER
PRIMARY KEY,
    Product TEXT NOT NULL,
    Category TEXT,
    Price REAL
);
```

Products.csv

EXERCISE

SQL JOINS - CANVAS

1. Create the Customers table

```
CREATE TABLE Customers (
    customer_id INTEGER PRIMARY KEY,
    customer_name TEXT NOT NULL
CHECK(length(customer_name) <= 100),
    email TEXT,
    city TEXT
);
```

Customers3.csv

2. Create the Orders table

```
CREATE TABLE Orders (
    order_id INTEGER PRIMARY
KEY,
    customer_id INTEGER,
    order_date TEXT,
    total_amount REAL,
    FOREIGN KEY
(customer_id) REFERENCES
Customers(customer_id)
);
```

Orders3.csv